

ATS Technical Documentation

Day 19 – ATS Documentation & Knowledge Transfer

1. Introduction

The ATS component of the Zecpath AI System is designed to support automated resume processing, candidate evaluation, scoring, ranking, and shortlisting.

The system combines structured resume processing, rule-based logic, semantic similarity analysis, configurable scoring, fairness controls, and API-based integration to produce explainable candidate evaluation results.

This document provides technical knowledge transfer for the ATS implementation developed during the internship work.

The purpose of this documentation is to make the system:

- Maintainable
 - Explainable
 - Testable
 - Configurable
 - Extensible
 - Easier for future developers to understand
-

2. High-Level ATS Workflow

The ATS processing workflow can be represented as:

Resume Input ↓ Text Extraction ↓ Text Cleaning ↓ Resume Normalization ↓ Section Classification ↓ Structured Information Processing ↓ Skill / Experience Analysis ↓ Semantic Matching ↓ ATS Scoring ↓ Fairness & Normalization Controls ↓ Candidate Ranking ↓ Shortlisting ↓ API / Recruiter-Friendly Output

Each stage performs a specific responsibility and provides structured information to subsequent processing stages.

3. Resume Input Processing

The ATS begins by processing candidate resume documents.

The parsing layer supports extraction and normalization operations before candidate evaluation.

Important components include:

- PDF reader
- DOCX reader
- Text cleaner
- Resume normalizer

- Resume section classifier
-

4. PDF Text Extraction

The PDF extraction component reads textual information from PDF resumes.

The optimized implementation processes the document page by page.

Extracted page content is stored temporarily and combined after processing instead of repeatedly extending one large string.

The processing flow is:

PDF Resume ↓ Open Document ↓ Iterate Through Pages ↓ Extract Page Text ↓ Collect Text ↓ Join Extracted Content ↓ Return Resume Text

The PDF resource is safely released after extraction.

5. Text Cleaning

Raw resume text may contain formatting noise introduced by document conversion or different resume templates.

The text-cleaning stage handles:

- Unicode normalization
- Multiple spaces
- Excessive blank lines
- Irregular line endings
- Control characters
- Bullet variations
- Incorrectly decoded bullet characters
- Decorative separators
- Common heading formatting

The goal is to provide cleaner input for downstream ATS processing.

6. Resume Normalization

The Resume Normalizer converts structured resume information into a consistent representation.

Normalization includes:

- Consistent key formatting
- Lowercase text normalization
- Whitespace normalization
- List normalization
- Duplicate removal
- Nested dictionary handling

- Null-value handling
- Configurable personal-attribute masking

Example:

Input:

Python Python SQL Python

Normalized output:

python sql

This reduces duplicate or inconsistent information before candidate evaluation.

7. Resume Section Classification

The Resume Section Classifier identifies logical resume sections using normalized heading recognition.

Supported section types include:

- Profile
- Skills
- Work Experience
- Education
- Certifications
- Projects

Multiple heading variants can map to the same standardized section.

For example:

Technical Skills Core Competencies Key Skills

can be recognized under the skills category.

8. Profile and Header Processing

Resume content appearing before the first recognized section commonly contains:

- Candidate name
- Current designation
- Professional title
- Contact header
- Profile information

The classifier treats this initial information as PROFILE rather than automatically placing it under UNKNOWN.

Example:

John Doe Data Scientist john@example.com

SKILLS Python SQL

is interpreted as:

PROFILE John Doe Data Scientist john@example.com

SKILLS Python SQL

This improves the representation of resume header information.

9. Skill Processing

The ATS processing pipeline supports structured extraction and evaluation of candidate skills.

Skill information can be compared with job requirements to determine a skill-match contribution.

The skill component forms one of the major inputs to the ATS scoring framework.

10. Experience Processing

Candidate experience information is processed to determine relevance to the target role.

Experience evaluation contributes to the ATS score through an experience-relevance value.

The system separates experience relevance from simple skill matching so that candidate suitability can be evaluated using multiple factors.

11. Semantic Matching

The Semantic Matching Engine evaluates contextual similarity between resume information and job-description information.

The implementation uses the SentenceTransformer model:

all-MiniLM-L6-v2

Semantic evaluation includes:

- Skills similarity
- Experience similarity
- Projects similarity

These values are combined into an overall semantic similarity score.

12. Semantic Processing Flow

The optimized semantic workflow processes:

Resume Skills Job Skills Resume Experience Job Experience Resume Projects Job Projects

as a batch.

The workflow is:

Six Text Inputs ↓ SentenceTransformer ↓ Normalized Embeddings ↓ Skills Pair Comparison Experience Pair Comparison Projects Pair Comparison ↓ Overall Semantic Similarity ↓ Match Classification

Batch encoding reduces unnecessary repeated model invocation during overall similarity calculation.

13. Semantic Model Reuse

The SentenceTransformer model is shared between Semantic Matching Engine instances within the same Python process.

The first engine instance loads the model when necessary.

Subsequent instances reuse the loaded model.

This avoids unnecessary repeated model initialization and improves repeated semantic-processing efficiency.

14. ATS Scoring Logic

Candidate evaluation is based on four major scoring parameters:

1. Skill Match
2. Experience Relevance
3. Education Alignment
4. Semantic Similarity

Each parameter contributes to the final ATS score according to the configuration of the target job role.

15. Weighted Scoring Formula

The overall ATS score is calculated using:

ATS Score = (Skill Match × Skill Weight) + (Experience Relevance × Experience Weight) + (Education Alignment × Education Weight) + (Semantic Similarity × Semantic Weight)

The result is rounded to produce an explainable candidate score.

16. Role-Specific Weight Configuration

Different job roles can use different scoring weights.

The current configuration includes support for roles such as:

Data Scientist

Skill Match: 0.40
Experience Relevance: 0.20
Education Alignment: 0.10
Semantic Similarity: 0.30

Frontend Developer

Skill Match: 0.35
Experience Relevance: 0.25
Education Alignment: 0.10
Semantic Similarity: 0.30

Backend Developer

Skill Match: 0.35
Experience Relevance: 0.30
Education Alignment: 0.10
Semantic Similarity: 0.25

Business Analyst

Skill Match: 0.35
Experience Relevance: 0.25
Education Alignment: 0.15
Semantic Similarity: 0.25

This makes scoring behavior configurable according to role requirements.

17. Explainable Score Output

The scoring engine returns both an overall score and its contributing values.

Example structure:

Job Role: Data Scientist

Overall Score: 85.0

Recommendation: Strong Candidate

Score Breakdown: Skill Match: 90 Experience Relevance: 80 Education Alignment: 75 Semantic Similarity: 85

This makes the scoring process easier to inspect than returning only a final number.

18. Candidate Recommendation

The scoring engine converts the overall score into a candidate recommendation.

The implemented recommendation bands are:

Score ≥ 90 Excellent Candidate

Score ≥ 80 Strong Candidate

Score ≥ 70 Good Candidate

Score ≥ 60 Average Candidate

Score < 60 Needs Improvement

These labels provide a human-readable interpretation of the ATS score.

19. Candidate Ranking

After candidate scores are generated, candidates can be sorted according to their overall scores.

The ranking engine assigns:

- Candidate score
- Rank
- Ordered candidate position

Candidates with higher ATS scores appear above candidates with lower scores.

20. Candidate Shortlisting

The shortlisting component categorizes ranked candidates using configurable thresholds.

The tested shortlisting configuration included:

Shortlist Threshold: 80

Review Threshold: 60

The resulting decisions include:

- Shortlisted
- Review
- Rejected

This provides recruiter-friendly output rather than requiring recruiters to interpret raw scores alone.

21. Fairness and Normalization

The ATS includes fairness-related controls designed to reduce inappropriate influence from non-essential personal information.

The implemented controls include:

- Resume normalization
- Personal-attribute masking
- Keyword score control
- Semantic contribution monitoring
- Bias-indicator reporting

Personal attributes configured for masking can be replaced with:

[MASKED]

before evaluation.

22. Fairness Indicators

The fairness evaluation can report indicators such as:

- Whether personal attributes were masked
- Whether keyword dependency was reduced
- Whether semantic contribution was sufficient
- Whether bias-related flags were detected
- Whether additional review is required

These controls improve transparency around automated candidate evaluation.

23. ATS System Testing

The ATS was evaluated using controlled candidate test cases across:

- Technical roles
- Non-technical roles
- Fresher resumes
- Senior profiles

Automated ATS decisions were compared with predefined manual review decisions.

The Day 17 controlled evaluation produced:

Total Cases: 12

Correct Predictions: 11

Incorrect Predictions: 1

Accuracy: 91.67%

Precision: 100.00%

Recall: 87.50%

F1 Score: 93.33%

One false-negative mismatch was identified.

These results represent the controlled development test dataset and should not be interpreted as production-wide real-world accuracy.

24. Mismatch Finding

The identified mismatch involved:

Role: Business Analyst

Profile: Non-Tech Fresher

ATS Score: 69.05

ATS Decision: REJECT

Manual Decision: SELECT

The candidate was 0.95 points below the system-level test selection threshold of 70.

This highlighted the need for future work on:

- Borderline review zones
 - Role-specific thresholds
 - Fresher-aware evaluation
 - Larger human-reviewed validation datasets
-

25. ATS API Layer

The ATS provides a REST API integration layer using FastAPI.

The API design supports ATS capabilities including:

- Health monitoring
- Resume parsing
- Candidate scoring
- Candidate shortlisting
- Validation handling
- Error handling
- Asynchronous job processing

The API allows ATS functionality to be consumed by backend systems without requiring those systems to directly manage internal processing modules.

26. API Response Design

API responses use structured output so that calling systems can distinguish:

- Request success/failure

- Message
- Returned data
- Error information

A successful scoring response can contain:

- Job role
- Overall score
- Recommendation
- Score breakdown

Standardized error information includes an error code and message.

27. Error Handling and Logging

Logging is used throughout the system to record important processing events.

Examples include:

- Engine initialization
- PDF extraction
- Resume normalization
- Semantic matching
- ATS scoring
- Processing failures

Error handling is designed to prevent individual processing failures from producing unclear system behavior.

28. Performance Optimization

Day 18 introduced performance and stability improvements including:

- More efficient PDF text construction
- Compiled text-processing patterns
- Improved noisy-resume cleaning
- Header/profile detection
- Shared semantic model reuse
- Batch semantic embedding generation
- Normalized embedding comparison

Measured post-optimization development results included:

Text Cleaning: 0.001457 sec

PDF Extraction: 0.105023 sec

Semantic Processing: 0.183752 sec

Second Semantic Engine Initialization: 0.001425 sec

Peak Tracked Python Allocation During the Specific Memory Test: 0.0040 MB

These measurements establish a development baseline rather than a complete production benchmark.

29. Regression Validation

After semantic optimization, the existing semantic matching test was rerun.

The result remained:

Skills Similarity: 76.63%

Experience Similarity: 66.39%

Projects Similarity: 84.05%

Overall Similarity: 75.69%

Match Level: Good Match

The existing semantic matching test passed successfully after optimization.

30. Configuration

Important ATS behavior is stored in configuration files rather than being entirely hardcoded.

Examples include:

- ATS role weight configuration
- Semantic similarity thresholds
- Shortlisting thresholds

This approach allows system behavior to be adjusted without rewriting the entire processing pipeline.

31. Maintainability

The ATS implementation separates responsibilities across components.

Examples include:

- Parsing
- Cleaning
- Normalization
- Classification
- Skill processing
- Experience processing
- Semantic matching
- Scoring

- Fairness
- Ranking
- API integration
- Testing

This modular separation makes individual components easier to test and maintain.

32. Extensibility

The ATS can be extended by:

- Adding new job-role weight configurations
- Adding new resume section-heading variants
- Improving entity extraction
- Expanding supported document types
- Adding larger evaluation datasets
- Calibrating role-specific thresholds
- Adding additional scoring factors
- Extending API endpoints
- Improving monitoring and performance benchmarking

Changes should be validated using regression tests before deployment.

33. Current Limitations

The current development implementation has several areas for future improvement.

These include:

- Larger real-world validation datasets
- More non-tech role coverage
- Better borderline-candidate handling
- Role-specific selection thresholds
- Expanded entity detection
- Image-based/OCR resume support where required
- Production load testing
- Concurrent request benchmarking
- Persistent monitoring
- Human-review calibration

These limitations should be considered before production deployment.

34. Conclusion

The ATS system provides a modular candidate evaluation pipeline covering resume processing, normalization, section classification, semantic matching, configurable scoring, fairness controls, ranking, shortlisting, API integration, testing, and performance optimization.

The implementation emphasizes explainability by exposing score components and configurable role weights rather than relying solely on an opaque final candidate score.

This technical documentation provides a knowledge-transfer reference for maintaining and extending the ATS implementation.